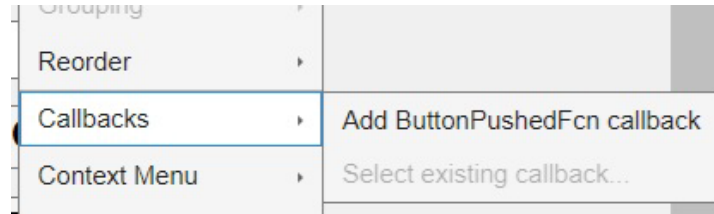


MEDIDOR DE VOLTAJE Y TEMPERATURA USANDO UNA PICO

M. en I. José Antonio de Jesús Arredondo Garza

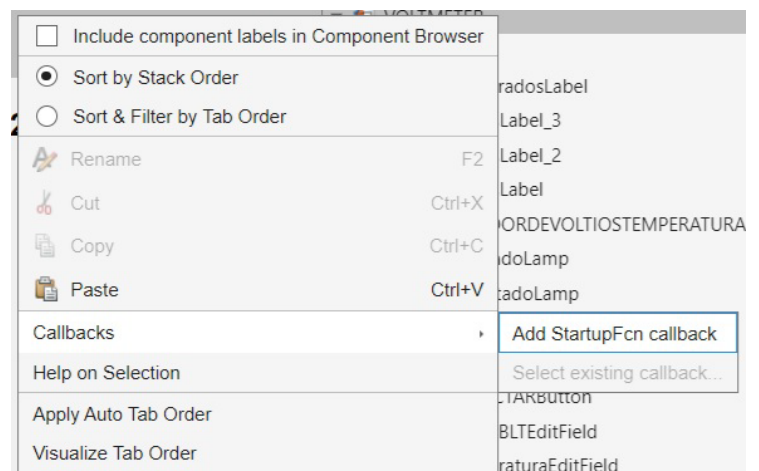
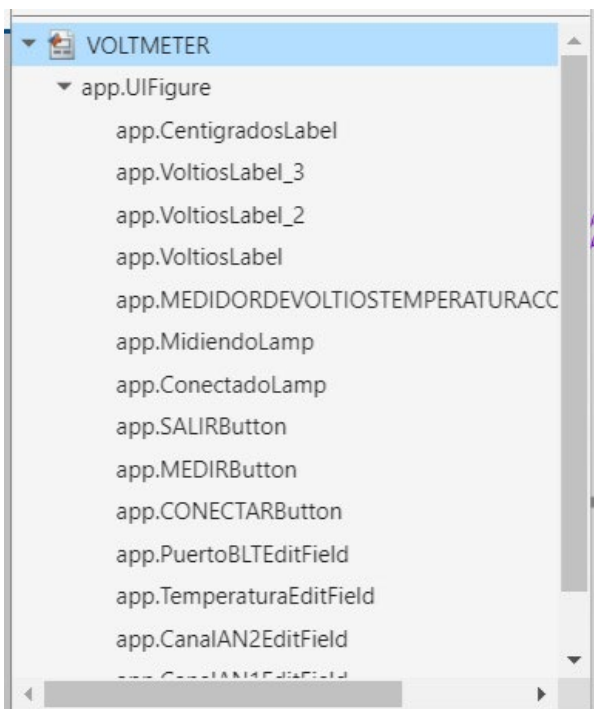
Para agregar código al botón SALIR haga click con el botón derecho en donde aparecerá lo siguiente:



Haga click en “**Add ButtonPushedF on callback**” en donde se desplegara el código del botón (que no es modificable). En el espacio en blanco teclee lo siguiente:

```
% Button pushed function: SALIRButton
function SALIRButtonPushed(app, event)
% XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX %
% CÓDIGO PARA SALIR DE LA APLICACIÓN DEL BOTÓN SALIR %
% XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX %
close(app.UIFigure)
clear all;
end
```

Ahora se va a crear la rutina de inicio del programa, que será responsable de la configuración inicial, para ello se hará click con el botón derecho sobre “VOLTMETER”. Luego vaya a “**callbacks**” y a donde dice “**Add StatupFcn callback**”.



Ahora se tiene la siguiente ventana de código y se necesita crear la función “*private property*” para crear las condiciones iniciales de las variables usadas:

```

% Properties that correspond to app components
properties (Access = public) ...

% Callbacks that handle component events
methods (Access = private)

% Code that executes after component creation
function startupFcn(app)

end

% Button pushed function: SALIRButton
function SALIRButtonPushed(app, event)
    % XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX %
    % CÓDIGO PARA SALIR DE LA APLICACIÓN DEL BOTÓN SALIR %
    % XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX %
    close(app.UIFigure)
    clear all;
end
end

```

Note que se está incluyendo el código del botón SALIR el cual, como se puede ver, es un código muy sencillo.

El código que se deberá escribir en “*properties*” será el siguiente:

```
properties (Access = private)
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%                                                                    %
%          UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO                %
%          FACULTAD DE INGENIERÍA                                  %
%          DEPARTAMENTO DE CONTROL Y ROBÓTICA                      %
%                                                                    %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%                                                                    %
%          M.I. JOSÉ ANTONIO DE JESÚS ARREDONDO GARZA              %
%          EMAIL: jarredon@unam.mx                                %
%          AÑO 2025                                                %
%                                                                    %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%                                                                    %
%  INICIALIZACIÓN DE LA VARIABLES USADAS EN EL PROGRAMA          %
%                                                                    %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%

NOMBRE;                    % Variable que contendrá el nombre del
                           % puerto bluetooth.
PUERTO;                    % Variable que contendrá el puerto
                           % "bluetooth" a usar.
CONECTADO;                 % Bandera para indicar si el bluetooth
                           % está conectado.
AN0;                      % Variable que contendrá la lectura
                           % del canal AN0.
AN1;                      % Variable que contendrá la lectura
                           % del canal AN1.
AN2;                      % Variable que contendrá la lectura
                           % del canal AN0.
GRADOS;                   % Variable que contendrá los grados
                           % centigrados medidos.
SIZE_DATA;                % Variable que contendrá los datos
                           % recibidos por el puerto bluetooth.

end
```

En el espacio “*startupFcn*” inicialice los Leds y la bandera de conexión de la siguiente manera:

```
% Code that executes after component creation
function startupFcn(app)

%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%                                                                    %
%  CONDICIONES DE INICIO DEL PROGRAMA, QUE EN ESTE CASO          %
%  SOLO SE ESTÁN PONIENDO LOS LEDs INICADORES EN ROJO Y          %
%  EL LETRERO DE ADVERTENCIA SOBRE EL DISPOSITIVO                %
%  BLUETOOTH.                                                      %
%                                                                    %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
app.ConectadoLamp.Color = "Red";    % Pon el Led de conexión
                                   % bluetooth en rojo.
app.MidiendoLamp.Color  = "Red";    % Pon el Led de medición
                                   % en color rojo.
app.CONECTADO           = 0;        % Pon bandera "CONECTADO"
                                   % en cero.

end
```

Ahora se va a crear el código del botón “**CONECTAR**” haciendo click con el botón derecho sobre el botón para crear el campo en donde podamos escribir el siguiente código:

```
% Button pushed function: CONECTARButton
function CONECTARButtonPushed(app, event)

%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%                                                                 %
% REALIZA CONEXIÓN DE PUERTO BLUETOOTH E INDICA DE QUE %
% TAMAÑO SON LOS DATOS A RECIBIR, PON LA BANDERA QUE %
% INIDCA QUE ESTÁ CONECTADO Y PON EL VERDE EL LED DE %
% "CONECTADO". %
%                                                                 %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%

app.SIZE_DATA = 16; % Se recibirán 4 números en
                    % formato "float" (en MATLAB
                    % se les llama "single").
                    % Estos se reciben en formato
                    % "little ENDIAN" y el programa
                    % los convertirá a "single".

if app.CONECTADO == 1

%*****%
% SI YA ESTÁ CONECTADO ENVÍA UN MENSAJE %
%*****%
    warndlg(['Ya está conectado !'], 'MENSAJE')

else
%*****%
% CONEXIÓN DE PUERTO BLUETOOTH %
%*****%
    app.NOMBRE = app.PuertoEditField.Value; % Obten el nombre
                                           % del puerto BLT.
    app.PUERTO = bluetooth(app.NOMBRE, 1); % Conecta puerto;
    app.ConectadoLamp.Color = "Green"; % Pon LED en verde.
    app.CONECTADO = 1; % Pon bandera de
                      % conectado.

end
end
```

De la misma manera que se generó el código del botón anterior ahora se va a generar el campo de código para el botón “MIDIENDO”.

```
% Button pushed function: MEDIRButton
function MEDIRButtonPushed(app, event)
    %XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
    %
    % EL CÓDIGO DEL BOTÓN PRIMERO DETECTA SI BLE ESTÁ CONECTADO %
    % Y SI ES EL CASO, TRANSMITE EL CARACTER DE SINCRONÍA "U" %
    % PARA QUE LA RASPBERRY PI PICO/PICO2 TRANSMITA TODAS SUS %
    % LECTURAS DEL CONVERTIDOR A/D Y CREA UNA PAUSA DE 0.125 SEG %
    % ENTRE LAS SECUENCIAS DE LECTURA. %
    %
    %XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
    if app.CONECTADO == 1

        for i = 1:100
            app.MidiendoLamp.Color = "Green"; % Pon LED en verde.

            Syncro = 85;
            write(app.PUERTO, Syncro, "uint8"); % Transmite el
                                                % caracter "U" de
                                                % sincronía que es
                                                % el 85.

            pause(0.1) % Pausa 0.1 Seg.

            %*****%
            % Lee el Puerto BLT y convierte los datos a "single" %
            % (que es un float). %
            %*****%
            % NOTA IMPORTANTE: se están recibiendo 16 bytes, %
            % pero como se están convirtiendo a "single", se %
            % reciben 4 "singles", por eso se pone SIZE_DATA %
            % dividido entre 4. %
            %*****%
            DATOS = read(app.PUERTO, app.SIZE_DATA/4, "single");
            flush(app.PUERTO)

            %*****%
            % SEPARA LOS DATOS OBTENIDOS %
            %*****%
            app.AN0 = DATOS(1);
            app.AN1 = DATOS(2);
            app.AN2 = DATOS(3);
            app.GRADOS = DATOS(4);

            %*****%
            % ESCRIBE A LOS CAMPOS DE VISUALIZACIÓN %
            %*****%
            app.CanalAN0EditField.Value = app.AN0;
            app.CanalAN1EditField.Value = app.AN1;
            app.CanalAN2EditField.Value = app.AN2;
            app.TemperaturaEditField.Value = app.GRADOS;
        end
        app.MidiendoLamp.Color = "Yellow"; % Pon LED en Amarillo.
        pause(1) % Retardo 1 Segundo.
        app.MidiendoLamp.Color = "Red"; % Pon LED en rojo.
    else

        %*****%
        % SI NO ESTÁ HABILITADO EL MÓDULO BLUETOOTH %
        % ENVÍA UN MENSAJE. %
        %*****%

        warndlg('EL MÓDULO BLUETOOTH NO ESTÁ CONECTADO !', ...
                'MENSAJE')
    end
end
```


La carátula final de acuerdo con este código es la siguiente:

MEDIDOR DE VOLTIOS - TEMPERATURA CON PICO / PICO2

Canal AN0:

[Voltios]

Canal AN1:

[Voltios]

Canal AN2:

[Voltios]

Temperatura:

[Centigrados]

Conectado



Midiendo



Puerto

*Nota: el ciclo de medición se repite
100 veces, por lo que, para continuar
se debe pulsar MEDIR de nuevo*

CONECTAR

MEDIR

SALIR

El código de la Raspberry Pi Pico/Pico2 en Micropython es el siguiente:

```
#####
#####
####
####
####  RUTINA QUE LEE LOS CANALES AN0, AN1, AN2 DEL CONVERTIDOR ANALÓGICO - DIGITAL
####  Y EL CANAL AN4 QUE CORRESPONDE AL MEDIDOR DE TEMPERTURA INTERNO.
####
####  LUEGO DE REALIZAR LAS LECTURAS, SE ESTÁ HACIENDO UNA TRANSMISIÓN DE LOS
####  UTILIZANDO LA UART0 DE LA RASPBERRY PI PICO A TRAVES DE UN MÓDULO BLUETOOTH
####  TIPO "HC05", SINCRONIZANDO LA TRANSMISIÓN DE DATOS SE REALIZA CON LA
####  RECEPCIÓN DEL CARACTER "U" (0x55).
####
#####
#####
####
####  UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO - FACULTAD DE INGENIERÍA
####  DEPARTAMENTO DE CONTROL Y ROBÓTICA
####
#####
#####
####
####  M. EN I. JOSÉ ANTONIO DE JESÚS ARREDONDO GARZA
####  EMAIL: jarredon@unam.mx
####
####  AÑO 2025
####
#####
#####
```

```
from machine import Pin, Timer
from time import sleep, sleep_ms, sleep_us
from machine import ADC, UART
import _thread
```

```
import struct
```

```
machine.freq(270000000)
```

```
# Importa librería de Pines.
# Importa librerías de retardos.
# Importa librerías del A/D y la UART.
# Librería para poder utilizar
# los 2 nucleos.
# Librería para manejo de datos
# de "float" a "Little ENDIAN" y de
# "Little ENDIAN" a "float".
# Hacer que la CPU trabaje a 270 Mhz.
```

```
#####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
#####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
#####
##### CONFIGURACIÓN DEL CONVERTIDOR A/D PARA PODER LEER LOS CANALES AN0, #####
##### AN1, AN2 Y AN3 (TEMPERATURA), LA GENERACIÓN DE UNA ONDA CUADRADA #####
##### Y LA CONFIGURACIÓN DE LA UART0 DE LA RASPBERRY PI PICO/PICO2. #####
#####
#####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
#####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
```

```
AN0      = ADC(Pin(26))      # Define el objeto ADC (GPI26) del canal AN0.
AN1      = ADC(Pin(27))      # Define el objeto ADC (GPI27) del canal AN1.
AN2      = ADC(Pin(28))      # Define el objeto ADC (GPI28) del canal AN2.
SENSOR   = ADC(4)            # Declara el Canal que mide la Temperatura.
Cte_to_Volt = (3.3/65535)    # Constante para conversión a Voltios Reales.
```

```
##xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx##
## Variable que contendrá los      ##
## los bytes "Little ENDIAN"      ##
## que se usarán para enviar      ##
## a través de la UART0          ##
##xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx##
En_Bytes = bytearray(16)
```

```
##*****##
## Declara UART0 a una tasa de 460800 Bauds, ##
## Tx = Pin0, Rx = Pin1. La UART0 será      ##
## conectada al Módulo Bluetooth HC05.      ##
##*****##
UART0    = UART(0, baudrate = 460800, tx = Pin(0), rx = Pin(1))
```

```
##*****##
## Pines Accesorio Utilizados y Frecuencia de la Onda Cuadrada ##
##*****##
P25      = Pin(25, Pin.OUT)    # Pin nativo de la tarjeta como testigo.
Pin3     = Pin(3, Pin.OUT)     # Pin de acción paralela (que generará una
                                # onda cuadrada).
FREQ     = 1                   # Pon la frecuencia a 1 Hz.
Period   = (1/FREQ)            # Obten el periodo.
Half_Period = (Period/2)       # Obten el semi periodo.
```



```
#####
####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX####
####
####          RUTINA PARA GENERAR UNA ONDA CUADRADA EN PARALELO          ####
####
####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX####
#####
def Onda_Cuadrada():
    while True:
        Pin3.on()          # Pon en "1" al Pin GPIO3 de laa RP2.
        sleep(Half_Period) # Retardo del semiperiodo.
        Pin3.off()         # Pon en "0" al Pin GPIO3 de la RP2.
        sleep(Half_Period) # Retardo del semiperiodo.

#####
####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX####
####
####          RUTINA QUE REALIZA LA LECTURA DE DATOS Y SU TRANSMISIÓN SINCRONIZADA          ####
####          UTILIZANDO EL PUERTO UART0 Y EL MÓDULO BLUETOOTH HC05.          ####
####
####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX####
#####

_thread.start_new_thread(Onda_Cuadrada,())          # Inicia el "thread" de la onda cuadrada.

while True:
    P25.off()          # Apaga el Led testigo.

    ##*****##
    ## ¿Hay algún byte pendiente? ##
    ## (En este caso Rx) ##
    ##*****##
    if UART0.any() > 0:
        LLAVE = UART0.read()          # Lee el caracter recibido o
                                      # caracteres recibidos.

        ##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##
        ##
        ## Investiga si se recibió un caracter "U" ##
        ##
        ##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##
        if LLAVE[0] == 85:
            P25.on()          # Enciende el indicador de
                              # transmisión.

            Read0 = AN0.read_u16()*Cte_to_Volt          # Lee el Canal AN0 del A/D.
            Read1 = AN1.read_u16()*Cte_to_Volt          # Lee el Canal AN1 del A/D.
            Read2 = AN2.read_u16()*Cte_to_Volt          # Lee el Canal AN2 del A/D.
            Valor_T = SENSOR.read_u16()*Cte_to_Volt      # Lee el Canal AN3 del A/D.
            GRADOS = 27 - (Valor_T - 0.706)/0.001721      # Obten la Temperatura de
                                                         # la CPU (centigrados).

            print(Read0, Read1, Read2, GRADOS)
```

```

##*****##
##  Conversión de los "floats" a formato      ##
##  "Little ENDIAN" para poder transmitir    ##
##  la información a MATLAB utilizando      ##
##  la UART0 vía Bluetooth HC05.            ##
##*****##
En_Bytes = bytearray(struct.pack("f",Read0))
En_Bytes = En_Bytes + bytearray(struct.pack("f", Read1))
En_Bytes = En_Bytes + bytearray(struct.pack("f", Read2))
En_Bytes = En_Bytes + bytearray(struct.pack("f", GRADOS))

##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##
##                                                                 ##
##      TRANSMITE LA CADENA DE BYTES OBTENIDOS      ##
##                                                                 ##
##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##

UART0.write(En_Bytes)

P25.off()          # Apaga el indicador de
                   # transmisión.

```

